

Prueba de Caja Blanca

“REGISTRO DE VIAJES TRANS-LARA IVAN EXPRESS”

Integrantes:

Kerlly Bonilla

Darwin Cuchipe

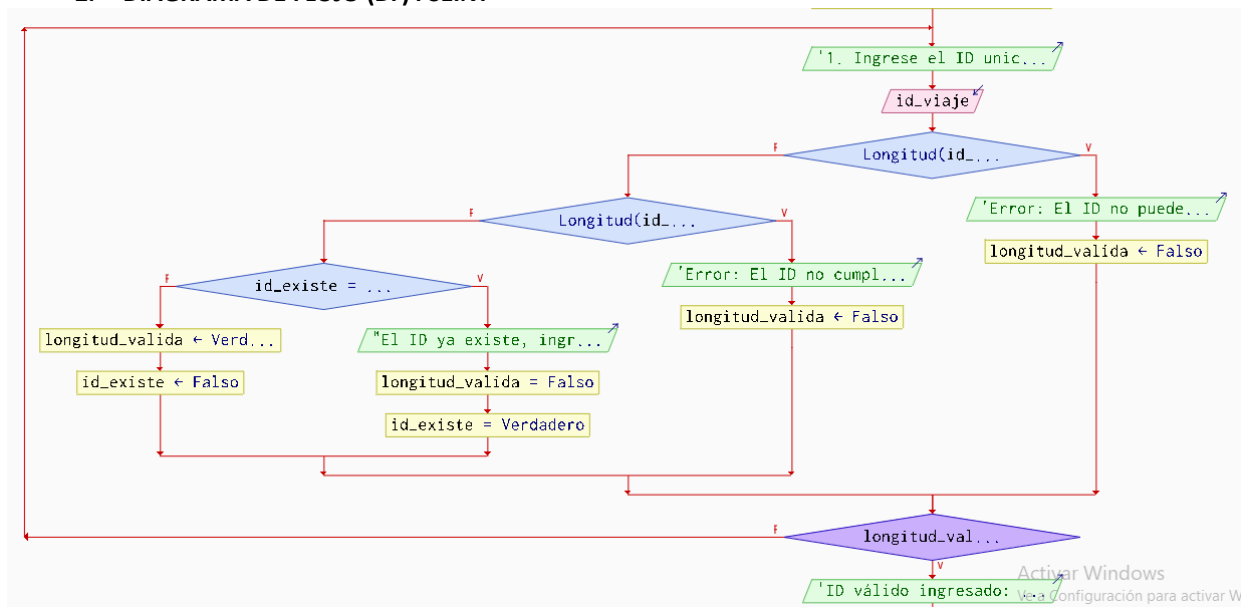
Fecha: 2025/07/22

Prueba caja blanca de Requisito N° 1: El programa deberá validar que el ID no sea vacío, cumpla estrictamente con 6 caracteres y que sea único antes de permitir su creación.

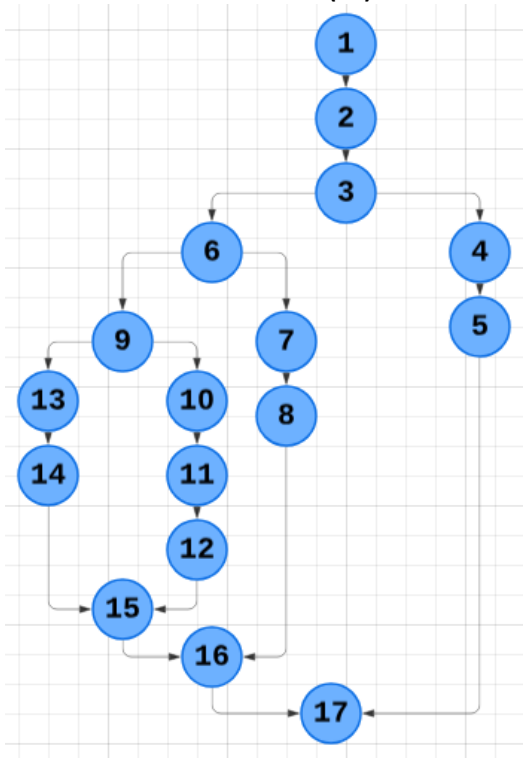
1. CÓDIGO FUENTE

```
do {
    printf("1. Ingrese el ID unico del viaje (ej. VJE001, maximo 6 caracteres):
");
    fgets(temp_input, sizeof(temp_input), stdin);
    temp_input[strcspn(temp_input, "\n")] = '\0';
    strcpy(r->id, temp_input);
    if (strlen(r->id) == 0) {
        printf("Error: El ID no puede estar vacio.\n");
    } else if (strlen(r->id) != 6) {
        printf("Error: El ID no cumple con los 6 caracteres. Ejemplo:
VJE001\n");
    } else if (buscarPorID(lista, cantidad, r->id) != -1) {
        printf("Error: El ID '%s' ya existe. Por favor, ingrese un ID
unico.\n", r->id);
    }
} while (strlen(r->id) == 0 || strlen(r->id) != 6 || buscarPorID(lista,
cantidad, r->id) != -1);
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) Prueba de

RUTAS:

R1: 1→2→3→4→5→17

R2: 1→2→3→6→7→8→16→17

R3: 1→2→3→6→9→10→11→12→15→16→17

R4: 1→2→3→6→9→13→14→15→16→17

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

6. $V(G) = \text{número de nodos predicados(decisiones)} + 1$
 $V(G) = 3 + 1 = 4$

7. $V(G) = A - N + 2$
 $V(G) = 19 - 17 + 2 = 4$

8.

DONDE:

P: Número de nodos predicado

A: Número de aristas

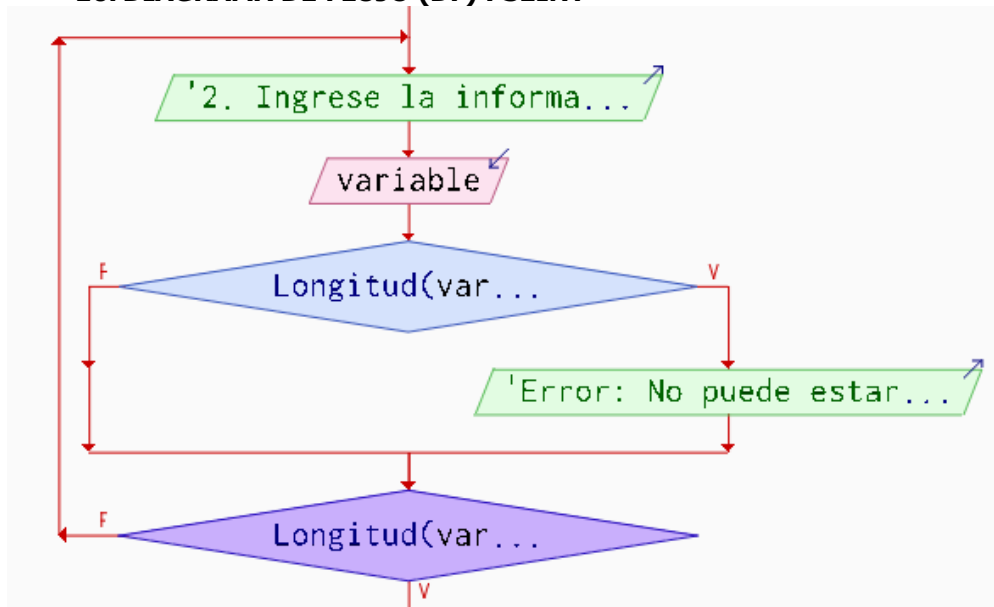
N: Número de nodos

Prueba caja blanca de Requisito N° 2: El programa deberá validar que la descripción de la carga, el nombre del conductor, el tipo de vehículo, la empresa, el origen, el destino y las observaciones no estén vacíos.

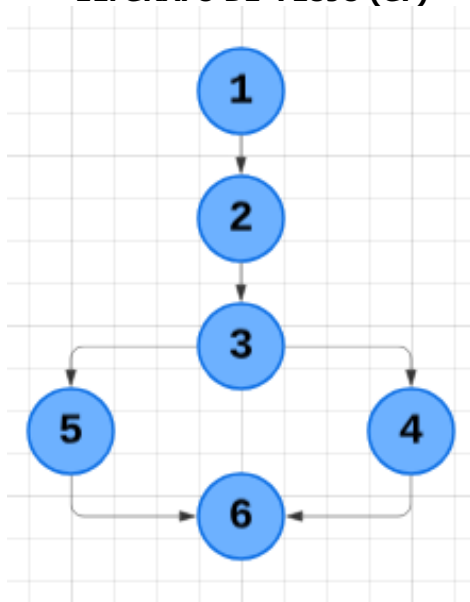
9. CÓDIGO FUENTE

```
// 2. Descripcion - VALIDACION: No vacia
do {
    printf("2. Ingrese la descripcion de la carga (maximo 200 caracteres):
");
    fgets(r->descripcion, sizeof(r->descripcion), stdin);
    r->descripcion[strlen(r->descripcion)] = '\0';
    if (strlen(r->descripcion) == 0) {
        printf("Error: La descripcion de la carga no puede estar
vacía.\n");
    }
} while (strlen(r->descripcion) == 0);
```

10. DIAGRAMA DE FLUJO (DF) PSEINT



11. GRAFO DE FLUJO (GF)



12. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1→2→3→4→6

R2: 1→2→3→5→6

13. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

14. $V(G) = \text{número de nodos predicados(decisiones)} + 1$ $V(G) = 1 + 1 = 2$

**15. $V(G) = A - N + 2$
 $V(G) = 6 - 6 + 2 = 2$**

DONDE:

P: Número de nodos predicado

A: Número de aristas

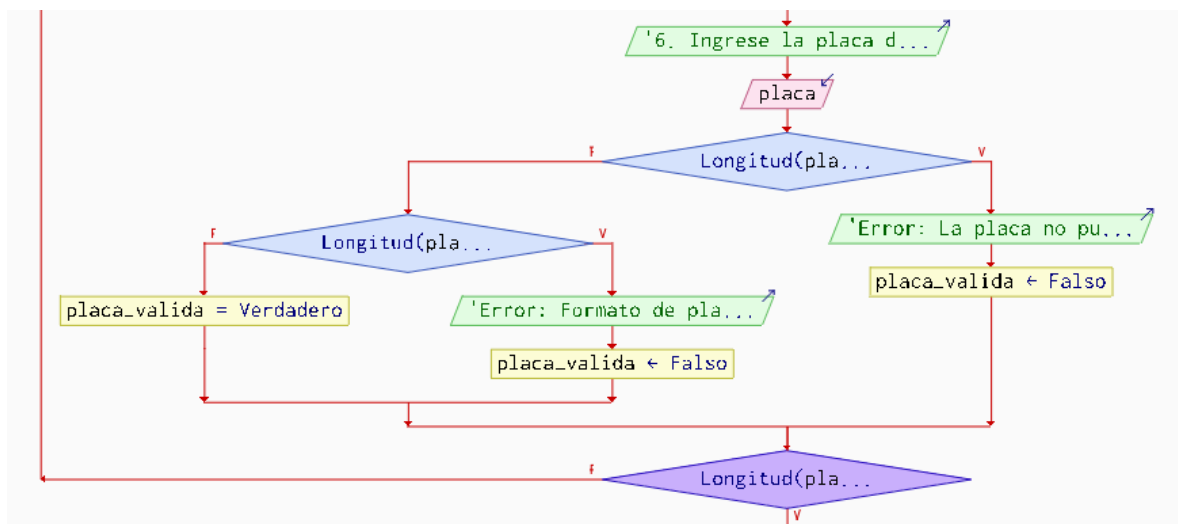
N: Número de nodos

Prueba caja blanca de Requisito N° 3: El programa deberá validar que la placa del vehículo cumpla con el formato "PPP-000" (tres letras mayúsculas, guion, cuatro dígitos).

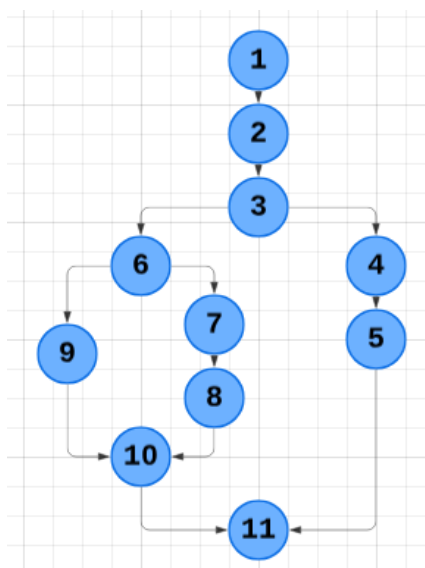
16. CÓDIGO FUENTE

```
do {
    printf("6. Ingrese la placa del vehiculo (Formato PPP-0000, maximo 8 caracteres): ");
    fgets(r->placa, sizeof(r->placa), stdin);
    r->placa[strcspn(r->placa, "\n")] = '\0';
    if (strlen(r->placa) == 0) {
        printf("Error: La placa no puede estar vacia.\n");
    } else if (!validarPlaca(r->placa)) {
        printf("Error: Formato de placa invalido. Use PPP-0000 (ej. ABC-123, XYZ-9876).\n");
    }
} while (strlen(r->placa) == 0 || !validarPlaca(r->placa));
```

17. DIAGRAMA DE FLUJO (DF) PSEINT



18. GRAFO DE FLUJO (GF)



19. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1→2→3→4→5→11

R2: $1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 10 \rightarrow 11$

R3: $1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 9 \rightarrow 10 \rightarrow 11$

20. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

21. $V(G) = \text{número de nodos predicados} + 1$ $V(G) = 2 + 1 = 3$

**22. $V(G) = A - N + 2$
 $V(G) = 12 - 11 + 2 = 3$**

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos

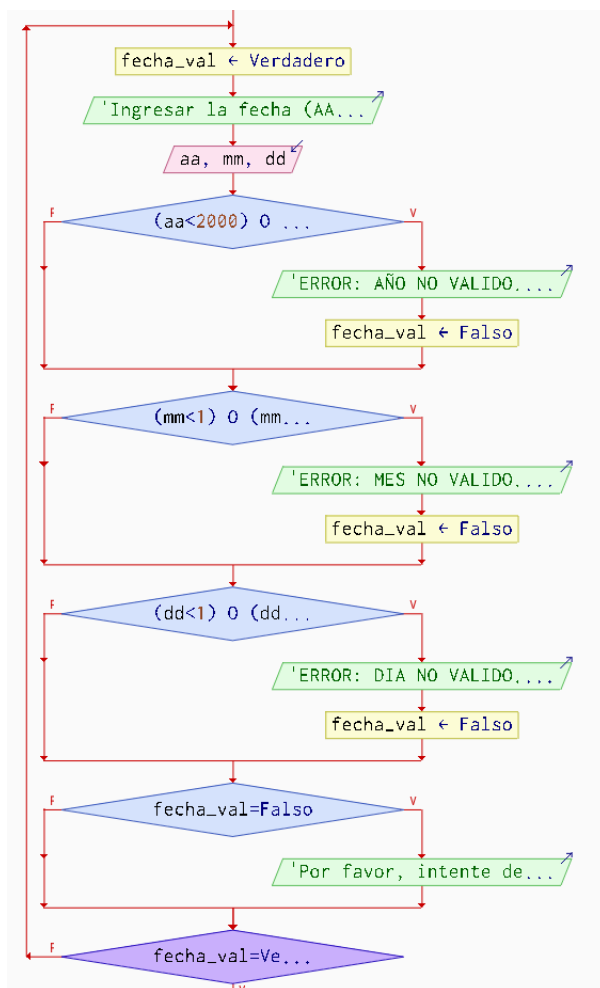
Prueba caja blanca de Requisito N° 4: El programa deberá validar que las fechas de salida y retorno cumplan con el formato "YYYY-MM-DD", que el año sea igual o posterior a 2000, que el mes esté entre 1 y 12, y que el día sea válido para el mes y año dados (incluyendo años bisiestos).

23. CÓDIGO FUENTE

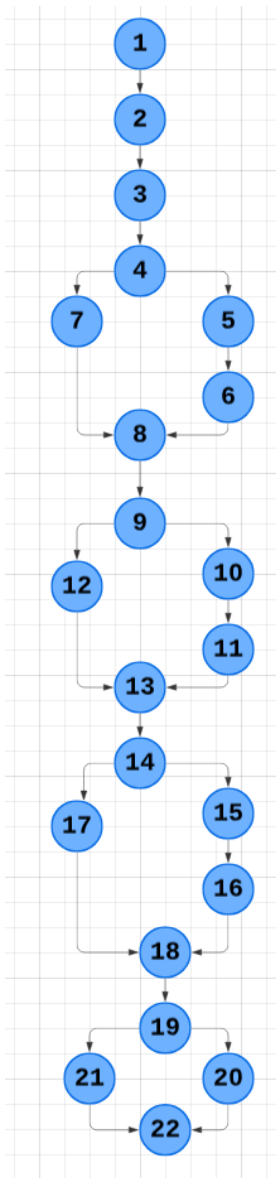
```
int validarFecha(const char *fecha) {
    int year, month, day;
    if (sscanf(fecha, "%d-%d-%d", &year, &month, &day) != 3) {
        return 0; // Formato incorrecto
    }
    if (year < 2000 || year>2025) {
        return 0; // Año menor a 2000 o mayor a 2025
    }
    if (month < 1 || month > 12) {
        return 0; // Mes fuera de rango
    }
    // Validar días según el mes
    int daysInMonth[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}; //
    0 es un placeholder

    if (day<1 || day > daysInMonth[month]) {
        return 0; // Día fuera de rango para el mes
    }
    return 1; // Fecha válida
}
```

24. DIAGRAMA DE FLUJO (DF) PSEINT



25. GRAFO DE FLUJO (GF)



26. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 13 → 14

R2: 1 → 2 → 3 → 4 → 5 → 6 → 7 → (8 → 9 → 10 → 11 → 12 → 13 → 14) → (vuelve a 1)

R3: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → (10 → 11 → 12 → 13 → 14) → (vuelve a 1)

R4: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → (12 → 13 → 14) → (vuelve a 1)

R5: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 13 → 14 → (vuelve a 1)

27. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

28. $V(G) = \text{número de nodos} + \text{predicados(decisiones)} + 1$ $V(G) = 4 + 1 = 5$

**29. $V(G) = A - N + 2$
 $V(G) = 25 - 22 + 2 = 5$**

DONDE:

P: Número de nodos predicado

A: Número de aristas

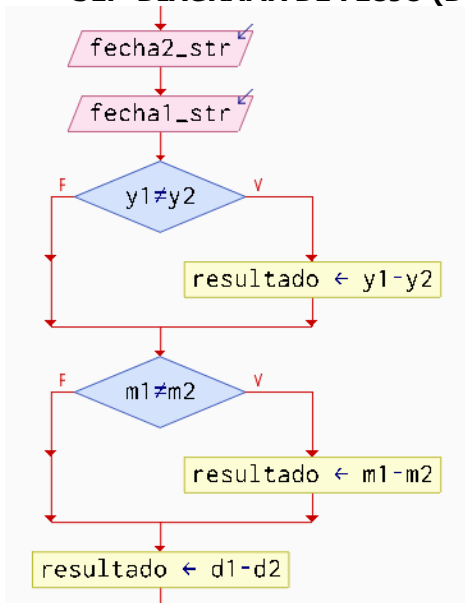
N: Número de nodos

Prueba caja blanca de Requisito N° 5: El programa deberá validar que la fecha de retorno no sea anterior a la fecha de salida.

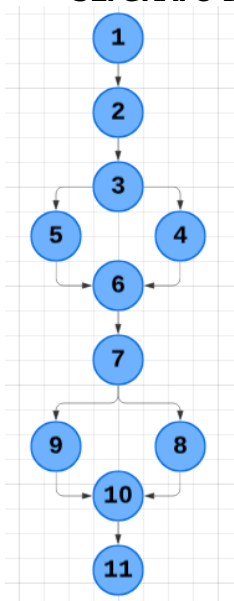
30. CÓDIGO FUENTE

```
int compararFechas(const char *fecha1_str, const char *fecha2_str) { int
    y1, m1, d1, y2, m2, d2; sscanf(fecha1_str, "%d-%d-%d", &y1, &m1, &d1);
    sscanf(fecha2_str, "%d-%d-%d", &y2, &m2, &d2);
    if (y1 != y2) return y1 - y2;
    if (m1 != m2) return m1 - m2;
    return d1 - d2;
}
```

31. DIAGRAMA DE FLUJO (DF) PSEINT



32. GRAFO DE FLUJO (GF)



33. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1→2→3→4→6→7→8→10→11

R2: 1→2→3→5→6→7→8→10→11

R3: 1→2→3→5→6→7→9→10→11

34. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

35. $V(G) = \text{número de nodos predi- cados(decisiones)} + 1$ $V(G) = 2 + 1 = 3$

**36. $V(G) = A - N + 2$
 $V(G) = 12 - 11 + 2 = 3$**

DONDE:

P: Número de nodos predi- cado

A: Número de aristas

N: Número de nodos

